

UNIVERSIDADE DO OESTE DE SANTA CATARINA

Campus Chapecó · Curso de Sistemas de Informação

25830 · Introdução ao Hardware · 2026/2

RELATÓRIO TÉCNICO

Cancela Automática de Estacionamento

Projeto Prático — Sensor + Atuador com Arduino

Atividade Avaliativa III

Bruno Henrique Siemer

Lucas Riva

Vitor Benetti

Ray Alvez

Professor: Jacson Luiz Matte

Chapecó, outubro de 2026

Resumo

Este projeto é uma cancela automática de estacionamento feita com Arduino Uno no Tinkercad. Um sensor ultrassônico HC-SR04 mede a distância até o veículo. Quando ela fica abaixo de 20 cm por três leituras seguidas, um micro servo SG90 levanta a cancela de 0° a 90°. Depois de 3 s sem veículo, a cancela fecha. Um LED verde e um vermelho mostram o estado, um buzzer bipa ao abrir e ao fechar, e o Monitor Serial mostra distância, estado e número de veículos. Leituras inválidas são ignoradas e, se um carro aparecer durante o fechamento, a cancela reabre.

Vídeo demonstrativo: https://www.youtube.com/watch?v=MZZckL_OgI8
Vídeo de apresentação: https://youtu.be/PbUeV_gl8yA?is=EGDlXcGaTXNvUtkf
Circuito no Tinkercad: <https://www.tinkercad.com/things/3RgD19BytQl-surprising-hillar>
Código-fonte completo: https://github.com/vitorsaugo/UNOESC/blob/main/2%C2%BA%20Semestre/Introdu%C3%A7%C3%A3o%20ao%20Hardware/TrabalhoFinal%20-%20S2/Cancela_Estacionamento.ino

1 Introdução e problema

Em estacionamentos pequenos, como de condomínios, empresas e da universidade, a cancela costuma ser aberta à mão por um porteiro ou por um botão. Isso gera fila e depende de alguém presente. O usuário do sistema é o motorista, que quer entrar sem esperar, e o responsável pelo estacionamento, que quer controlar e contar os acessos.

Objetivo: abrir e fechar a cancela automaticamente a partir da distância medida até o veículo, avisando o motorista com luz e som.

2 Pesquisa: projetos de referência

- **Projeto de cancela automática com Arduino: guia completo:** cancela com Arduino, sensor ultrassônico e servo motor. *Aproveitamos* o mesmo princípio: detectar a aproximação do veículo e movimentar a cancela. *Acrescentamos* LEDs para indicar o estado da cancela, buzzer para sinalização e confirmação por mais de uma leitura do sensor (3 leituras seguidas).
- **Automatic Barrier Gate With ESP32 and HC-SR04 (Instructables):** mesma ideia com ESP32. *Mudamos:* usamos Arduino Uno e adaptamos o funcionamento ao nosso projeto, com movimento suave do servo, contador de veículos e um fechamento que verifica se ainda existe veículo próximo, para a cancela não fechar enquanto o carro está passando.

3 Componentes utilizados

Componente	Função	Ligação	Qtde	Obtenção
Arduino Uno R3 + cabo USB	executa o programa	–	1	do curso
Sensor HC-SR04	mede a distância até o veículo (2 a 400 cm)	TRIG D9 · ECHO D10 · VCC 5V · GND	1	do curso
Micro servo SG90	levanta (90°) e abaixa (0°) a cancela	sinal D6 · 5V · GND	1	do curso
LED verde	cancela aberta	D4 + resistor	1	do curso
LED vermelho	cancela fechada	D5 + resistor	1	do curso
Resistor 220 Ω	limita a corrente dos LEDs	em série com cada LED	2	do curso
Buzzer piezo	bipe ao abrir e ao fechar	+ D3 · – GND	1	do curso
Protoboard e jumpers	distribui 5V e GND	trilhos + e –	1	do curso

Tabela 1: Lista de materiais.

4 Montagem do circuito, passo a passo

1. No Tinkercad, criamos um circuito novo com Arduino Uno R3 e uma protoboard.
2. Ligamos o **5V** do Arduino ao trilho + e o **GND** ao trilho – da protoboard. Depois interligamos os trilhos de cima com os de baixo, para que todos os componentes tivessem o mesmo 5V e o mesmo GND.
3. **Sensor HC-SR04:** VCC no trilho +, GND no trilho –, TRIG no pino D9 e ECHO no pino D10.
4. **Servo SG90:** fio vermelho no +, marrom no – e laranja (sinal) no pino D6.
5. **LEDs:** cada LED recebeu um resistor de 220 Ω em série. O pino D4 vai ao LED verde e o D5 ao vermelho; a perna negativa (cátodo) de cada um vai ao trilho –.
6. **Buzzer piezo:** terminal + no pino D3 e terminal – no trilho –.
7. Colamos o código na aba *Código* (modo Texto) e iniciamos a simulação.

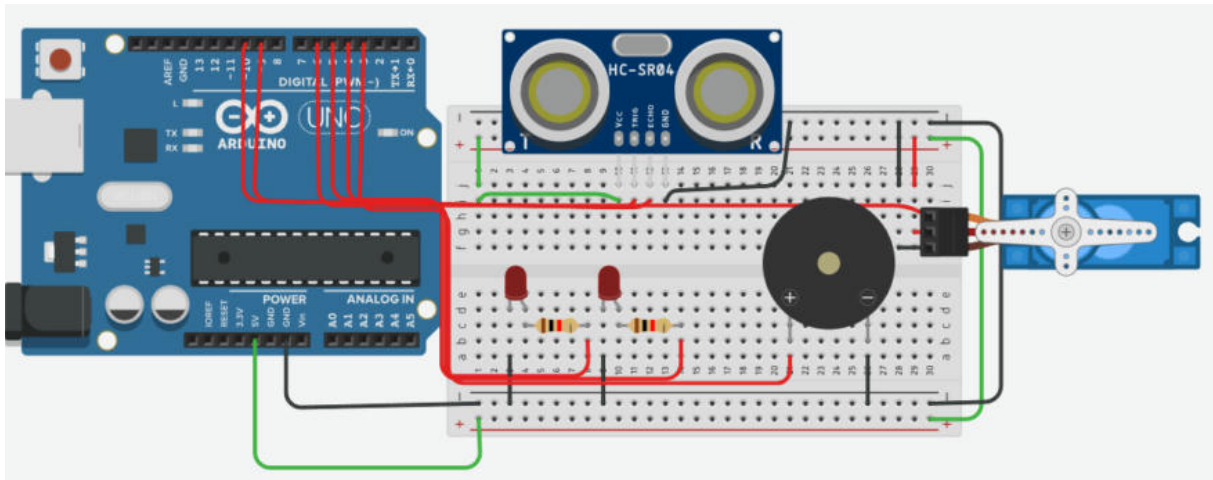


Figura 1: Circuito montado no Tinkercad.

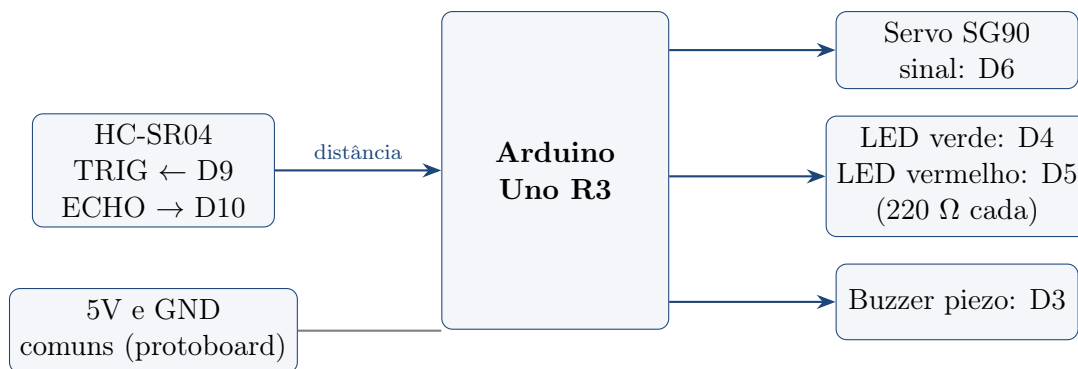


Figura 2: Ligações com os pinos usados no código.

5 Funcionamento da cancela, passo a passo

1. **Ao ligar (autoteste):** o LED verde acende por 0,4 s, depois o vermelho por 0,4 s, e o buzzer dá 1 bipe. Isso confirma que LEDs e buzzer estão ligados. Em seguida a cancela fica fechada (servo em 0°) com o LED vermelho aceso.
2. **Leitura do sensor:** a cada 100 ms, o Arduino manda um pulso de 10 μ s no TRIG. O sensor emite o ultrassom e o pino ECHO fica em nível alto até o eco voltar. A distância é calculada por $d = t \times 0,0343/2$ (velocidade do som em cm/ μ s, dividida por 2 porque o som vai e volta).
3. **Leitura inválida:** se não houver eco em 30 ms ou a distância ficar fora de 2–400 cm, a leitura é descartada e nada muda.
4. **Carro chegando:** cada leitura abaixo de 20 cm soma 1 num contador; uma leitura acima de 20 cm zera o contador. Com 3 leituras seguidas (cerca de 0,3 s), a cancela abre.

5. **Abertura:** o LED verde acende e o vermelho apaga, o buzzer dá 1 bipe e o servo sobe de 0° a 90° , 1° a cada 15 ms (movimento suave). O contador de veículos aumenta 1 e aparece no Monitor Serial.
6. **Cancela aberta:** enquanto o carro continuar a menos de 20 cm, o cronômetro de saída é reiniciado com `millis()`. A cancela não fecha com o carro embaixo.
7. **Fechamento:** depois de 3 s seguidos sem carro, o buzzer dá 2 bipes e o servo desce de 90° a 0° . Ao terminar, o LED verde apaga e o vermelho acende.
8. **Segurança:** durante a descida, a cada 10° , o sensor é lido de novo. Se houver carro, a cancela para, dá 1 bipe e volta a 90° , sem contar um veículo novo.

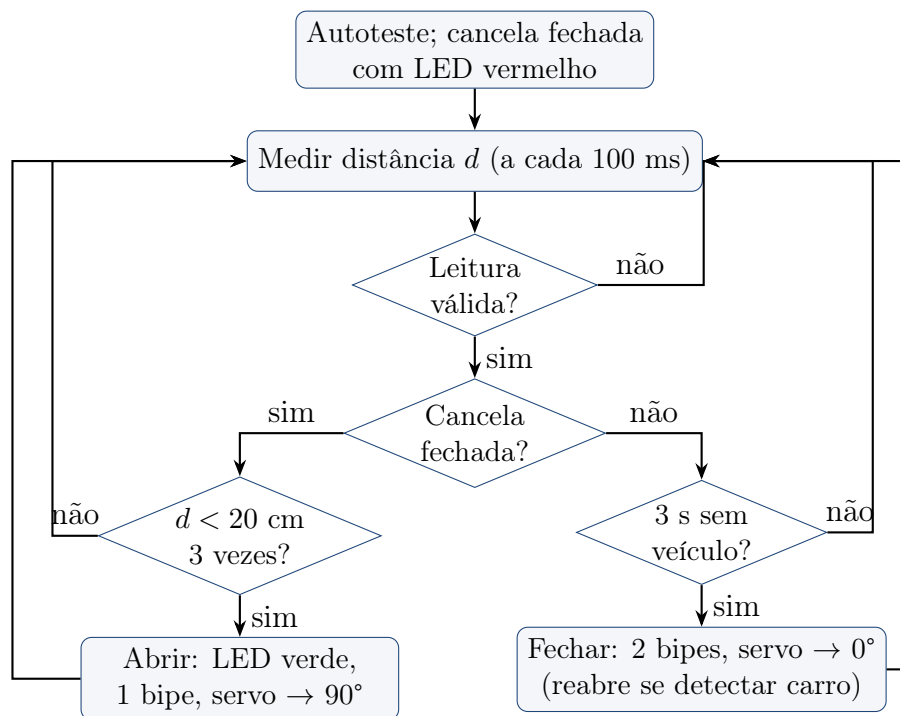


Figura 3: Fluxograma do funcionamento.

6 Código: trechos principais

O código completo está no link do resumo. Os pinos e os parâmetros (limiar de 20 cm, 3 leituras, 3 s, ângulos) ficam em constantes no início, e cada ação é uma função: `medirDistancia()`, `abrirCancela()`, `fecharCancela()`, `bipe()`, `ledsAberta()` e `ledsFechada()`.

6.1 Leitura do sensor

O tempo limite de 30 ms no `pulseIn()` impede que o programa trave sem eco. Toda leitura inválida retorna `-1` e é ignorada.

```

1 float medirDistancia() {
2   digitalWrite(PINO_TRIG, LOW);

```

```

3   delayMicroseconds(2);
4   digitalWrite(PINO_TRIG, HIGH);
5   delayMicroseconds(10);
6   digitalWrite(PINO_TRIG, LOW);
7
8   unsigned long duracao = pulseIn(PINO_ECHO, HIGH, 30000UL);
9   if (duracao == 0) return -1;
10
11  float distancia = duracao * 0.0343 / 2.0;
12  if (distancia < 2 || distancia > 400) return -1;
13  return distancia;
14 }

```

6.2 Regra de decisão

É aqui que o sensor comanda o atuador: fechada, abre com 3 leituras seguidas abaixo de 20 cm; aberta, só fecha quando millis() indica 3 s sem veículo.

```

1  bool perto = d < DISTANCIA_LIMITE_CM;
2
3  if (!aberta) {
4      leiturasPerto = perto ? leiturasPerto + 1 : 0;
5      if (leiturasPerto >= LEITURAS_CONFIRMACAO) {
6          abrirCancela();
7      }
8  } else {
9      if (perto) {
10         ultimoCarroVisto = millis();
11     } else if (millis() - ultimoCarroVisto >= TEMPO_FECHAR_MS) {
12         fecharCancela();
13     }
14 }

```

6.3 Fechamento com reabertura

A cancela desce 1° por vez. A cada 10°, o sensor é consultado; se houver carro, ela volta a abrir.

```

1  while (anguloAtual > ANGULO_FECHADA) {
2      anguloAtual--;
3      cancela.write(anguloAtual);
4      delay(PASSO_SERVO_MS);
5
6      if (anguloAtual % 10 == 0 && carroPresente()) {
7          Serial.println("!! Veiculo detectado durante fechamento -
8              reabrindo");
9          bipe(1);
10         moverServo(ANGULO_ABERTA);
11         ledsAberta();
12         ultimoCarroVisto = millis();
13         return;
14     }
15 }

```

```

13 }
14 }

```

7 Testes e ajustes

Problema encontrado: no checkpoint, o sensor e o servo funcionavam, mas os LEDs e o buzzer não. A causa era o GND do Arduino não ligado ao trilho – da protoboard, onde os LEDs e o buzzer estavam aterrados. Ligamos o GND ao trilho, interligamos os trilhos de cima e de baixo e tudo passou a funcionar. Depois disso criamos o autoteste, que mostra esse tipo de falha logo ao ligar.

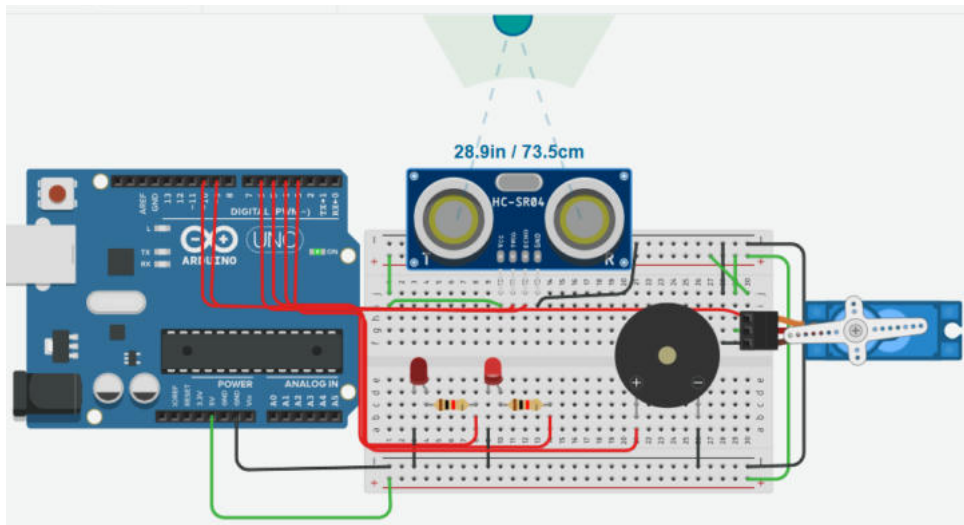


Figura 4: Simulação: objeto a 73,5 cm, cancela fechada e LED vermelho aceso.

Teste	Esperado	Obtido
Iniciar a simulação	verde, vermelho, 1 bipe; depois vermelho aceso	OK
Objeto a 73,5 cm	cancela fechada, LED vermelho	OK
Objeto a 10 cm	1 bipe, LED verde, servo a 90°, veículos +1	OK
Objeto a 19 cm / 21 cm	abre / não abre	OK
Objeto parado embaixo da cancela	não fecha	OK
Objeto sai	após 3 s: 2 bipes, servo a 0°, LED vermelho	OK
Objeto volta durante o fechamento	reabre, não soma veículo	OK

Tabela 2: Registro de testes.

Calibração: o limiar de 20 cm foi escolhido para a escala de maquete (numa cancela real seria cerca de 1 m). As 3 leituras de confirmação (cerca de 0,3 s) evitam abrir por um reflexo isolado. Os 3 s de espera dão tempo para o carro passar, e 15 ms por grau deixam o movimento suave (cerca de 1,35 s para abrir ou fechar).

8 Resultados e melhorias

O sistema faz o ciclo completo sensor → decisão → atuador no Tinkercad: abre com o carro presente, sinaliza com LEDs, buzzer e serial, fecha só depois que o carro sai e reabre se ele voltar durante o fechamento.

Limitações: o movimento do servo e os bipes usam `delay()`. Enquanto a cancela abre (cerca de 1,6 s), o sensor não é lido. Durante o fechamento, o LED verde continua aceso até o servo chegar a 0°.

Com mais uma semana: trocar esses `delay()` por `millis()`, montar o circuito na bancada com uma maquete e mostrar o contador de veículos num display LCD.

9 Divisão de tarefas e etapas do trabalho

Integrante	O que fez
Bruno Henrique Siemer	Montagem do circuito no Tinkercad (Arduino, protoboard, sensor, servo, LEDs, buzzer); diagnóstico e correção da falha de GND dos LEDs e do buzzer; integração do código com o circuito.
Lucas Riva	Código de leitura do HC-SR04 (<code>medirDistancia()</code>), tratamento de leituras inválidas e regra de decisão (3 leituras para abrir, 3 s para fechar).
Vitor Benetti	Código do servo (movimento suave e reabertura), dos LEDs e do buzzer; autoteste; execução dos testes e calibração.
Ray Alvez	Redação do relatório no Overleaf, gravação e edição dos vídeos demonstrativo e de apresentação.

Tabela 3: Divisão de tarefas.

Etapas realizadas pelo grupo:

1. **Escolha e pesquisa:** escolha da cancela de estacionamento no banco de ideias e pesquisa de dois projetos de referência.
2. **Proposta (29/09):** definição do problema, dos componentes, da regra de decisão e da divisão de tarefas.
3. **Montagem:** circuito montado no Tinkercad.
4. **Programação:** leitura do sensor, regra de decisão, servo, LEDs, buzzer e serial.
5. **Checkpoint (02/10):** sensor lendo na serial e servo respondendo; correção da ligação de GND.
6. **Testes e calibração:** testes da tabela acima e ajuste dos parâmetros.
7. **Entrega (09/10):** relatório, código e vídeos.

Referências

- *Projeto de cancela automática com Arduino: guia completo*. Disponível em: <https://www.tinkercad.com/things/3RgD19BytQl-surprising-hillar>. Acesso em: out. 2026.
- INSTRUCTABLES. *Automatic Barrier Gate With ESP32 and HC-SR04*. Disponível em: <https://www.instructables.com/Automatic-Barrier-Gate-With-ESP32-and-HC-SR04-Ardu/>. Acesso em: out. 2026.
- ARDUINO. *Language Reference e Servo Library*. Disponível em: <https://www.arduino.cc/reference>. Acesso em: out. 2026.
- ELECFREAKS. *Ultrasonic Ranging Module HC-SR04* (datasheet).
- TOWER PRO. *SG90 Micro Servo* (datasheet).
- ANTHROPIC. *Claude* (IA generativa). Usada como apoio na revisão do código, no diagnóstico da fiação e na redação do relatório; o conteúdo foi revisado pelo grupo.